СТРАТЕГІЯ ТЕСТУВАННЯ

Від «перевіряю очима» до контракту: Jest, doubles, DI-моки, Vitest

Заняття №10

Node.js Pro

Лектор
Кирило Орлов

ДРАБИНКА ДОВІРИ: 8 КРОКІВ — ОЧІ → РАННЕР → DI → КОНТРАКТ

До перерви — кроки 1–4: раннер і doubles	Після перерви — кроки 5–8: чистота і контракт
крок 1 — біль: очі, float і assert без diff	крок 5 — fixtures і фабрики: arrange без шуму
крок 2 — Jest: AAA і межа рівно 1000 грн	крок 6 — coverage: чому 100% ≠ довіра
крок 3 — doubles руками: fake · stub · spy	крок 7 — контракт: toEqual і снапшоти
крок 4 — Nest: overrideProvider, мок через DI	крок 8 — Vitest: той самий тест, інший раннер

НАСКРІЗНА НИТКА
Тестовним сервіс робить не фреймворк, а КОНСТРУКТОР: залежності приходять ззовні — у тесті на їхні місця стають doubles. Драбинка веде від «повірив оку» до «поведінку зафіксовано контрактом».

ПЛАН ЗАНЯТТЯ

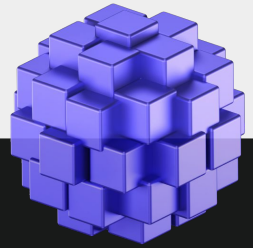


- Біль без раннера: очі, float, assert без diff
- Перший юніт: Jest, AAA, межі + піраміда
- Doubles руками → мок через DI у Nest
- Фабрики · coverage і його брехня · снапшоти
- Vitest + межі лекції · Тест #10 (2 бали)

КРОК 1: ЛОГІКА Є — РАННЕРА НЕМАЄ

- orderTotal() готовий; «тести» — console.log і звірка очима
- 100.10 + 200.20 = 300.29999999999995 — float-гривні брешуть
- node:assert: падає на ПЕРШІЙ невдачі — без diff, решта невідома
- межа «рівно 1000 грн» губиться першою: око звіряє «приблизно»

Потрібен раннер: усі кейси щоразу, diff при падінні, exit code для CI. Гроші — в копійках, не float.



```
ОКО (npm run step1)

очікували   отримали
  900.00      900.00  v
 1080.00     1080.00  v
 1000.00     1000.00  v?
…після 20 рядків око
звіряє «приблизно»


node:assert

✗ впав на ПЕРШІЙ
  перевірці, без diff
решта перевірок
не виконалась узагалі
```

КРОК 2: ТУ САМУ ЛОГІКУ ЗВІРЯЄ РАННЕР

```
// test/02-pricing.spec.ts — кожен тест за патерном AAA
import { OrderItem, orderTotal } from '../src/01-pricing';

describe('orderTotal — перший юніт (крок 2)', () => {
  it('сумує позиції без знижки до порога', () => {
    // Arrange
    const items: OrderItem[] = [
      { sku: 'BLT-1', title: 'Ремінь',
        priceKop: 45_000, qty: 2 },
    ];
    // Act
    const totalKop = orderTotal(items);
    // Assert
    expect(totalKop).toBe(90_000);
  });

  it('межа: рівно 1000 грн — знижки НЕМАЄ', () => {
    // граничні кейси губляться при «перевірці очима»
    const items = [{ sku: 'BAG-2', title: 'Сумка',
                     priceKop: 50_000, qty: 2 }];
    expect(orderTotal(items)).toBe(100_000);
  });

  it('округлює знижку вниз до цілої копійки', () => {
    // 100_001 * 0.9 = 90_000.9 → 90_000
    expect(orderTotal([{ sku: 'BAG-3', title: 'Сумка',
      priceKop: 100_001, qty: 1 }])).toBe(90_000);
  });
});
```

Одне it — одна поведінка, назва як речення

Arrange — підготував стан і дані

Act — викликав РІВНО одну дію

Assert — звірив результат із очікуванням

межа «рівно 1000 грн» — окремим it

npm run step2 → Tests: 5 passed

ЩО САМЕ КУПУЄ РАННЕР

З JEST

- усі кейси — щоразу, за пів секунди
- виконуються ВСІ it, не до першого падіння
- Expected / Received + стрілка на рядок
- exit code ≠ 0 — CI червоний

БЕЗ РАННЕРА

- запускаєш і звіряєш очима
- assert падає на першій невдачі
- diff-у немає — шукай сам
- CI не знає, зелено чи ні

СТРАТЕГІЯ: ПІРАМІДА ТЕСТІВ

юніт: одна одиниця логіки, все чуже — підмінено

інтеграційний: справжні БД/HTTP — повільніше, чесніше

e2e: весь застосунок як чорна скринька — їх одиниці

чим нижче — тим швидше і точніше вказує на рядок

здорова форма: багато юнітів, менше інтеграційних

сьогоднішня лекція живе на нижньому поверсі

```
      ПІРАМІДА ТЕСТІВ

        /   e2e   \        одиниці · хвилини
       /-----------\       весь шлях користувача
      / integration \      справжні БД і HTTP
     /---------------\     → L#16
    /      unit       \    сотні · мілісекунди
   /-------------------\   СЬОГОДНІ МИ ТУТ

перевернув → «ріжок морозива»:
повільний CI · флейкі · страх правок
```



МЕЖА ЮНІТА: СПРАВЖНІЙ РЕПОЗИТОРІЙ

CheckoutService: repo, gateway, clock приходять У КОНСТРУКТОР — це вже DI, без фреймворка

npm run step3:pain → ✗ Postgres на 127.0.0.1:21010 недоступний (ECONNREFUSED) — за секунду

тест, якому потрібна інфраструктура, — вже НЕ юніт: після L#5 підняти compose ви вмієте, але це «дорого і не про те» → integration-тест, L#16

Date.now() усередині — теж залежність: акцію «−5% у вихідні» чесно тестуєш двічі на тиждень; тому Clock — у конструкторі



TEST DOUBLES: ТАКСОНОМІЯ ЗАМІН

Double	Що робить	У нашому коді
dummy	заповнює параметр; ніхто його не викликає	—
stub	віддає заготовлену відповідь: керуємо ВХОДОМ	StubClock(FRIDAY)
spy	stub + журнал викликів: перевіряємо ВИХІД	SpyPaymentGateway.charges
fake	спрощена, але РОБОЧА реалізація	FakeOrdersRepo (Map)
mock	заготовлені очікування взаємодії	jest.fn() у кроці 4

```ts
// test/support/doubles.ts — doubles РУКАМИ: магії всередині немає

export class FakeOrdersRepo implements OrdersRepo {    // FAKE: Map
  private readonly rows = new Map<string, Order>();
  constructor(seed: Order[] = []) { /* seed → rows, ключ = order.id */ }
  async findById(id: string) { return this.rows.get(id) ?? null; }
  async save(order: Order) { this.rows.set(order.id, order); }
}

export class StubClock implements Clock {              // STUB: керуємо входом
  constructor(private readonly fixed: Date) {}
  now(): Date { return this.fixed; }
}

export class SpyPaymentGateway implements PaymentGateway {  // SPY: журнал
  readonly charges: number[] = [];
  async charge(amountKop: number) {
    this.charges.push(amountKop);
    return { txId: `tx_${this.charges.length}` };
  }
}

// test/03-doubles.spec.ts: субота −5% — будь-якого дня
const service = new CheckoutService(repo, gateway, new StubClock(SATURDAY));
await service.checkout('ord_2');
expect(gateway.charges).toEqual([95_000]);  // spy бачив суму зі знижкою
```



«КРАШ-ТЕСТ»: TEST DOUBLES ЯК АНАЛОГІЯ

ЧАСТИНА ТЕСТУ	АНАЛОГІЯ
• тестований сервіс — CheckoutService • doubles на місцях залежностей • StubClock — зафіксований час • SpyPaymentGateway пише виклики • прогін за мілісекунди, щоразу	• авто на випробувальному стенді • манекени замість пасажирів • стенд задає умови точно • датчики в манекені пишуть удар • не чекаємо справжньої аварії

Стенд доводить безпеку крісла/авто, а не те, що доїде до Києва. Юніт із doubles не ловить болю справжньої траси — стиків із живими Postgres і HTTP. Полігон — L#16.



```
// test/04-testing-module.spec.ts — мок через DI: штатна операція графа

const moduleRef = await Test.createTestingModule({
  imports: [CheckoutModule],  // модуль ЯК У ПРОДІ: PgOrdersRepo, HttpPaymentGateway
})
  .overrideProvider(ORDERS_REPO)
  .useValue(new FakeOrdersRepo([seedOrder]))  // ← замість Postgres
  .overrideProvider(PAYMENT_GATEWAY)
  .useValue(gateway)                          // ← spy замість справжніх грошей
  .overrideProvider(CLOCK)
  .useValue(new StubClock(FRIDAY))            // ← п'ятниця у будь-який день
  .compile();                                 // граф збирається ПІСЛЯ підміни

const service = moduleRef.get(CheckoutNestService);
const paid = await service.checkout('ord_1');

expect(paid.status).toBe('paid');
expect(gateway.charges).toEqual([108_000]);   // 1200 грн −10% знижки кошика

// npm run step4 → Tests: 3 passed (override / providers-у-тесті / jest.fn)
```



ПІДМІНА — ЧЕРЕЗ DI, А НЕ jest.mock

jest.mock(шлях)

підміняє МОДУЛЬ на рівні резолвера; тест зав'язується на шляхи файлів

overrideProvider

підміняє ТОКЕН у графі до compile(); модуль — той самий, що в проді

providers у тесті

міні-модуль прямо в тесті: ті самі токени, useValue — лаконічно

Чому це важливо
Залежності описано токенами ORDERS_REPO · PAYMENT_GATEWAY · CLOCK, тому підміна — штатна операція DI-графа. jest.mock за всю лекцію не знадобився жодного разу: клас навіть не знає, що його тестують.



```
// test/05-factory.spec.ts — фабрики: arrange без шуму

// «ДО»: у файлі 12 рядків (тут стиснуто) — важливе число тоне
const order = {
  id: 'ord_1',
  items: [{ sku: 'BAG-1', title: 'Сумка',
            priceKop: 150_000, qty: 1 }],
  status: 'new' as const,
};

// «ПІСЛЯ»: лишилось тільки суттєве
const order2 = makeOrder({ items: [makeItem({ priceKop: 150_000 })] });

// test/support/factories.ts — валідний об'єкт за замовчуванням:
export function makeOrder(overrides: Partial<Order> = {}): Order {
  seq += 1;
  return { id: `ord_${seq}`, items: [makeItem()],
           status: 'new', ...overrides };
}
// npm run step5 → Tests: 5 passed
```

FIXTURES І ФАБРИКИ: ПРАВИЛА ГРИ

01. fixture — НАЗВАНА константа зі змістом для команди

02. фабрика генерує валідне; override — лише суттєве

03. дефолти в ОДНОМУ місці — нове поле правиш раз

04. arrange худне: 12 рядків шуму → 1 рядок суті

05. спільного mutable-стану між тестами НЕМАЄ

COVERAGE: 100% — А БАГ ЖИВИЙ

```
$ npm run step6      (тести ганяють БАГОВАНУ версію shippingFee)

File                  | % Stmts | % Branch | % Funcs | % Lines
06-shipping-buggy.ts  |     100 |      100 |     100 |     100

Tests: 9 passed — усе зелене. А тепер сам баг:

  if (subtotalKop > 100_000) return 0;      // ← має бути >=
  банер обіцяє: «безкоштовна доставка ВІД 1000 грн»
  кошик РІВНО 100_000 коп → мінус 4_000 коп за доставку

  три «беззубі» тести пройшли обидві гілки і обидва плечі:
  expect(shippingFeeBuggy(...)).toBeGreaterThanOrEqual(0)

coverage міряє «рядок ВИКОНАВСЯ», а не «поведінку ПЕРЕВІРЕНО»
сильний тест цілиться в МЕЖУ: it.each([[100_000, 0], [99_999, 4_000]])
```

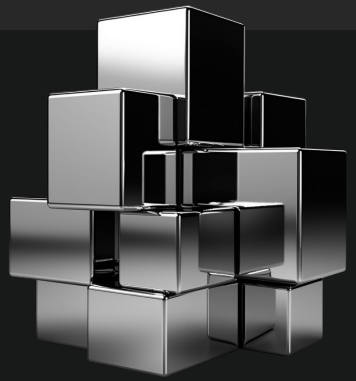


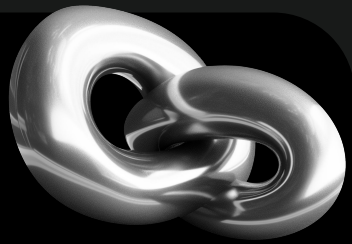
ЩО МОКАТИ, А ЩО ЛИШАТИ СПРАВЖНІМ

Мокай МЕЖІ СВІТУ: мережа, БД, час, чужі API — усе повільне й недетерміноване приходить через конструктор і підміняється

НЕ мокай те, що тестуєш: домен (orderTotal, CheckoutService) — справжній. Мок домену = тест перевіряє сам себе

⚠ Over-mocking: замокали все — тест зелений, прод падає. Більше моків = менше правди; чесна перевірка стиків — L#16





```
// test/07-contract.spec.ts — тест ФІКСУЄ публічний контракт

it('явний контракт: toEqual фіксує КОЖНЕ поле', () => {
  expect(orderNotFoundProblem('ord_7')).toEqual({
    type: 'https://api.example.ua/problems/order-not-found',
    title: 'Order not found',
    status: 404,
    detail: 'Замовлення ord_7 не існує',
    instance: '/orders/ord_7',        // problem+json — формат із L#9
  });
});

it('контракт снапшотом: еталон у __snapshots__/', () => {
  expect(orderNotFoundProblem('ord_7')).toMatchSnapshot();
  // 1-й запуск ЗАПИСУЄ еталон, наступні — звіряють
  // перейменував поле → падіння з diff; оновити еталон: jest -u (свідомо!)
  // у CI: jest --ci — відсутній снапшот = падіння, а не мовчазний запис
});

// + третій it: toMatchInlineSnapshot — еталон прямо в коді тесту (2-й снапшот)
// npm run step7 → Tests: 3 passed, Snapshots: 2 passed
```

JEST 30 ПРОТИ VITEST 4: ЧЕСНА ТАБЛИЦЯ

Аспект	Jest 30	Vitest 4
API	describe/it/expect — глобальні	ті самі, але через import
TS-трансформація	прошарок ts-jest поверх tsc	ESM-first, esbuild
Nest-екосистема	Jest-first: docs і schematics	не Nest-нативний: свій конфіг + tsconfig
Наші виміри	28 тестів ≈0.5 с · спек із нуля ≈0.5–0.7 с	2 тести ≈0.1 с · з нуля ≈0.5–0.6 с
Watch-режим	перезапуск через трансформер	без ts-jest; на великих базах відрив росте

```
// test-vitest/08-pricing.vitest.ts — ТОЙ САМИЙ тест під Vitest
// різниця з test/02-pricing.spec.ts — рядок імпорту: у Jest ці API глобальні
import { describe, expect, it } from 'vitest';
import { OrderItem, orderTotal } from '../src/01-pricing';

describe('orderTotal під Vitest (крок 8)', () => {
  it('сумує позиції без знижки до порога', () => {
    const items: OrderItem[] = [
      { sku: 'BLT-1', title: 'Ремінь', priceKop: 45_000, qty: 2 },
    ];
    expect(orderTotal(items)).toBe(90_000);
  });
  // + другий it: знижка НАД порогом і НЕ рівно на порозі
});

// $ npm run step8
//   RUN v4.1.10 …
//   Test Files  1 passed (1)  |  Tests  2 passed (2)  |  Duration ≈100 ms
// vitest.config.mts — саме .mts: пакет CommonJS, а конфіг Vitest — ESM
```

ЩО СВІДОМО ЛИШАЄМО ЗА ДУЖКАМИ

справжній Postgres у контейнері — testcontainers → L#16

supertest, HTTP-рівень і e2e Nest-застосунку → L#16

контракти між сервісами (Pact) → L#16

jest.mock і підміна модулів — нам вистачило DI

mutation testing (Stryker) — чесніша метрика, ніж coverage

бенчмарк-війни раннерів — міряй на СВОЄМУ репо

ПІДСУМКИ + МІСТ ДО L#11 І L#16

ДРАБИНКА

раннер → doubles → DI-мок → фабрики → контракт: кожна сходинка прибирає одну причину не довіряти зеленому

DI

тестовність дає КОНСТРУКТОР: залежності ззовні — у тесті замість них doubles. Nest робить підміну штатною

далі

L#11 — конфіги й секрети: env теж залежність · L#16 — інтеграційні: testcontainers, supertest, Pact

Тест #10

2 бали · піраміда · AAA · мокати чи справжнє · doubles · Jest vs Vitest — дедлайн у LMS

COVERAGE

міряє «рядок виконався», а не «поведінку перевірено». 100% ≠ немає багів

Q&A

???

r_d

ЗАВЖДИ Є КУДИ ЗРОСТАТИ